# Plagiarism Checker for Short Documents

*Project Report submitted for the partial fulfilment of grade for the course*

**Data Structures and Algorithms -3 (25CS2103E)**

**in**

**A.Y. 2026-2027**

Submitted by
Komali Varnika (2520090010)
K. Aasritha sai Lakshmi (2520090006)

Team Number: 27

Section: 12

Under the guidance of

Ms. K. Chandusha

Asst. Prof., CSIT

Hyderabad-500090, Telangana, India.



**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**




# DECLARATION

The Mini Project Report entitled "**Plagiarism Checker for Short Documents**" is a record of Bonafide work of **Komali Varnika – 2520090010**, **K. Aasritha sai Lakshmi – 2520090006**, submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**Name – Roll.No.**

**Komali Varnika – 2520090010**

**K. Aasritha sai Lakshmi – 2520090006**

## K L (Deemed to be) University

## DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY




## CERTIFICATE

This is to certify that the mini project based report entitled "**Plagiarism Checker for Short Documents**" is a bonafide work done and submitted by **Komali Varnika – 2520090010**, **K. Aasritha sai Lakshmi – 2520090006** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

**Signature of the Supervisor**

**Signature of the HOD** **Signature of the External Examiner**

## ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide, **Ms. K. Chandusha, Asst. Prof., CSIT**, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

**Name – Roll.No.**

**Komali Varnika – 2520090010**

**K. Aasritha sai Lakshmi – 2520090006**

## TABLE OF CONTENTS


| S.No. | Contents | Page no. |
|---|---|---|
| 1. | Abstract | 6 |
| 2. | Introduction | 7 |
| 3. | Literature survey | 8 |
| 4. | Problem Statement and Objectives | 10 |
| 5. | Hardware and Software requirements | 11 |
| 6. | Implementation | 12 |
| 7. | Results & Discussion | 18 |
| 8. | Conclusion & Future Scope | 21 |
| 9. | References | 22 |

# 1.ABSTRACT

This project presents a Plagiarism Checker for Short Documents with a Java back end and a browser front end. Students sign up or log in with their name and roll number and then choose one of two modes. In Web check mode a submitted document is compared with an index of web sources and the engine reports the overall match percentage, the sites ranked by similarity and the most similar site. In Compare mode two uploaded documents are compared with each other. PDF, Word (.docx), PowerPoint (.pptx) and text files are accepted, or text can be pasted. The text is extracted in the browser and sent to a Java server, which splits it into words and maps every word to an integer ID. A Rabin-Karp rolling hash over three-word windows finds candidate shared phrases, and each candidate is verified with the Knuth-Morris-Pratt algorithm (Web check) or the Z-function (Compare). For documents that pass this filter, the two word sequences are joined with separators, a suffix array is built using prefix doubling, and Kasai's algorithm computes the Longest Common Prefix (LCP) array; adjacent suffixes with long common prefixes expose every shared passage of four or more consecutive words. The result is shown as a match percentage with the similar words highlighted, and every user has a saved history of past checks. The interface uses a pastel design with a light and dark mode switch and a 3D cube and tilting cards. The algorithms are written in Java using only the standard library, and the front end is written in HTML, CSS and JavaScript.

**Team Name:** Team 27

**Team Members:**

1. Komali Varnika, 2520090010
2. K. Aasritha sai Lakshmi, 2520090006

# 2. INTRODUCTION

Plagiarism in academic writing has grown alongside the easy availability of online material. Students can copy a passage, change a few words and submit it as their own. Short documents such as essays, abstracts and lab reports are common targets, because a few copied sentences can make up a large part of the whole text.

Checkers that compare whole files or look only for identical sentences are easily fooled by this kind of light editing. Commercial tools such as Turnitin handle it better, but they are black boxes: the user sees a similarity percentage without understanding how it was computed or exactly which words matched.

This project builds a **plagiarism checker for short documents** with a Java back end and a browser front end. Documents in PDF, DOCX, PPTX or plain-text form are converted to text in the browser and sent to a Java server, which splits them into words and maps each word to an integer ID. Candidate shared phrases are found with a Rabin-Karp rolling hash and verified with KMP or the Z-function. A **suffix array** with Kasai's **LCP array** then finds every run of matching words in near-linear time. Results are shown as a match percentage and as highlighted text, so that the exact copied words are visible.

The platform has two modes. Web check compares a submitted document with a set of web sources and ranks the sources by similarity, so the user can see which site is most similar. Compare checks two uploaded documents against each other. Users sign up or log in with their name and roll number and can reopen the summary of earlier checks from a history page. The interface offers a pastel theme, a light and dark mode switch and a 3D cube and tilting cards.

The rest of this report is organised as follows: Section 3 surveys related work, Section 4 states the problem and objectives, Section 5 lists hardware and software requirements, Section 6 presents the implementation, Section 7 discusses results, Section 8 concludes with future scope, and Section 9 lists references.

# 3. LITERATURE SURVEY

This section reviews earlier systems and techniques that are directly related to our work. Each is summarised together with how it influenced the design of this project.

**1. Winnowing and MOSS (Schleimer, Wilkerson, Aiken, 2003).** Winnowing selects the minimum hash in each sliding window of k-gram hashes, which guarantees that any long enough shared substring produces a common fingerprint while keeping the fingerprint set small. It is the basis of the MOSS system used widely for source-code similarity. Our project instead locates exact matches with a suffix array, which gives precise word positions for highlighting rather than only fingerprints.

**2. MOSS: Measure Of Software Similarity (Stanford).** MOSS is a free web service that compares programs and reports matching regions between submissions. It shows that automated pairwise comparison works well for coursework, but it is aimed at source code, not natural-language prose. We apply pairwise comparison with highlighted matching regions to ordinary text documents in PDF, Word and PowerPoint formats.

**3. COPS copy-detection prototype (Brin, Davis, Garcia-Molina, 1995).** COPS was one of the first systems for detecting copies of digital documents. It registers documents in a database and checks new text sentence by sentence, using hashing to find overlaps. It is effective for verbatim sentence copying but weak against small edits within a sentence. Our engine reports every shared run of four or more consecutive words, so a copied phrase is found even when the rest of the sentence has been changed.

**4. Suffix arrays (Manber and Myers, 1993).** Manber and Myers introduced the suffix array as a space-efficient alternative to suffix trees, built by prefix doubling in O(n log n) time. It supports fast substring search and is a standard tool for finding repeated substrings. We build a suffix array over integer word IDs of two documents joined by a sentinel to find their common word sequences.

**5. Linear-time LCP computation (Kasai et al., 2001).** Kasai and co-authors showed that the longest-common-prefix array of a suffix array can be computed in O(n) time using the inverse suffix array. Adjacent LCP values directly expose repeated substrings. In our engine each adjacent pair of suffixes that comes from different documents and has an LCP of four or more words is marked as a direct match.

**6. Rabin-Karp rolling hash (Karp and Rabin, 1987).** Karp and Rabin proposed comparing a pattern with a text through hash values instead of character by character, and showed that the hash of the next window can be obtained from the previous one in constant time. Equal hashes

can still be false matches, so they must be verified. We hash every three-word window of a document with a rolling hash to find candidate shared phrases quickly.

**7. Knuth-Morris-Pratt algorithm (Knuth, Morris and Pratt, 1977).** KMP finds a pattern in a text in O(n + m) time by precomputing a failure table, so the text pointer never moves backward. We use it to verify that a candidate phrase found by hashing really occurs in a web source.

**8. Z-function (Gusfield, 1997).** The Z-function gives, for every position of a string, the length of the longest substring starting there that matches a prefix of the string. Applied to a pattern, a separator and a text, it finds every occurrence in linear time. We use it to verify candidate phrases between two uploaded documents.

**9. Document resemblance by shingling (Broder, 1997).** Broder proposed representing a document as a set of overlapping word sequences called shingles and measuring resemblance as the overlap between two sets. This set-overlap view is cheap and tolerant of small edits. We use the same idea on a small scale: three-word windows of a document are hashed with Rabin-Karp to find candidate overlaps with a source.

**10. Turnitin similarity reports.** Turnitin is a widely used commercial service that compares submissions with a large database and returns a similarity percentage with colour-coded highlights. It is powerful but closed and does not explain the algorithm. Our tool provides a similar highlighted view and a ranking of the most similar sources, but uses transparent, well-known algorithms that are explained in this report.

**11. Client-side document parsing (pdf.js, JSZip).** Open-source libraries make it possible to read documents in the browser without a server. pdf.js from Mozilla extracts text from PDF pages, and JSZip opens the zipped XML inside .docx and .pptx files so that the text of every paragraph or slide can be collected. We combine these two libraries so that users can compare documents in their original formats.

# 4. Problem Statement and Objectives

**Problem Statement:**

To build a web application with a Java back end that checks a short document against web sources and compares two uploaded documents, showing the match percentage and highlighting the similar text.

**Problem Description:** Manual checking of a document against web sources or against another document is slow, and simple tools cannot show which exact words match or which source is most similar. The checker must accept common file formats, report a clear match percentage and highlight the matching text.

## Objectives:

### Identify the Project's Purpose:

The project was started to apply string algorithms from this course (Rabin-Karp, KMP, Z-function, suffix arrays and LCP arrays) to a practical problem and to produce an easy-to-use plagiarism checker for essays, abstracts and reports. The objectives are:

- Extract text from PDF, DOCX, PPTX and plain-text files in the browser.
- Split documents into words and map each word to an integer ID in Java.
- Find candidate shared phrases with a Rabin-Karp rolling hash and verify them with KMP or the Z-function.
- Find all shared passages of four or more consecutive words between two texts using a suffix array and Kasai's LCP array.
- In Web check mode, report the overall match percentage and rank the sources by similarity to show the most similar site.
- In Compare mode, show how much of each document appears in the other and display both documents with highlighted matches.
- Provide sign up and login with name and roll number, a saved history of past checks, a light and dark mode switch and a pastel 3D interface.

# 5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher
- RAM: 4 GB minimum (8 GB recommended for large PDF files)
- Storage: 100 MB free disk space

Software Requirements:

- Operating System: Windows 10/11, Linux or macOS
- Language: Java (JDK 11 or newer) for the back end; HTML5, CSS3 and JavaScript for the front end
- Libraries: Java standard library only (including com.sun.net.httpserver); in the browser, pdf.js 3.11.174 and JSZip 3.10.1 (loaded from CDN)
- Browser: latest Chrome, Safari, Edge or Firefox (uses localStorage); editor: Visual Studio Code

# 6. IMPLEMENTATION

The application has a Java back end and a single-page front end. The back end (Server.java and Algorithms.java) uses only the Java standard library and its built-in HTTP server. The front end (index.html) is written in HTML, CSS and JavaScript: it extracts text from the uploaded file, posts it to the server and shows the returned percentages and highlighted words. The core of the system is a text-processing pipeline made of the modules below. Table 6.1 summarises them.

**Table 6.1** Modules of the plagiarism checker

| # | Module / function | Purpose | Complexity |
|---|---|---|---|
| 1 | Text extraction (readFile, browser) | Reads PDF, DOCX, PPTX, TXT/MD into plain text | O(file size) |
| 2 | Tokenizer and word IDs (Server.prep) | Splits text into words and maps each word to an integer ID | O(L) |
| 3 | Rabin-Karp, KMP, Z-function (Algorithms) | Rolling-hash candidate search and exact verification of phrases | O(n + m) each |
| 4 | Suffix array, LCP and match (Algorithms) | Prefix-doubling suffix array, Kasai LCP array, marks shared passages of 4+ words | O(n log² n) + O(n) |
| 5 | Web check and compare engines (Server) | Ranks sources by similarity, compares two documents, returns JSON | O(n log² n) per source or pair |
| 6 | Login and history (index.html) | Name and roll number accounts, per-user history in localStorage | O(1) per operation |

**Module 1: Multi-format Text Extraction**

Plain-text files are read with the browser File API. Word (.docx) and PowerPoint (.pptx) files are opened as ZIP archives with JSZip, and the text of every paragraph (w:p) or slide paragraph (a:p) in the XML is collected. PDFs are read page by page with pdf.js and the text items are joined. The extracted text is then posted to the Java server.

```
async function readFile(f) {
  const e = f.name.split('.').pop().toLowerCase();
  if (['txt', 'md', 'csv', 'html', 'json'].includes(e)) {
    let t = await f.text();
    return e == 'html' ? new DOMParser().parseFromString(t, 'text/html').body.textContent :
t;
  }
  if (['docx', 'pptx', 'odt'].includes(e)) {
    const z = await JSZip.loadAsync(await f.arrayBuffer());
    let out = '';
    const names = Object.keys(z.files).filter(n =>
      e == 'docx' ? n == 'word/document.xml'
      : e == 'pptx' ? /^ppt\/slides\/slide\d+\.xml$/.test(n)
      : n == 'content.xml')
      .sort((a, b) => a.localeCompare(b, undefined, {numeric: true}));
```

```
    for (const n of names)
      out += (await z.file(n).async('string'))
        .replace(/<\/(w:p|a:p|text:p)>/g, '\n').replace(/<[^>]+>/g, ' ') + '\n';
    return out.replace(/&amp;/g, '&').replace(/&lt;/g, '<').replace(/&gt;/g, '>')
              .replace(/&quot;/g, '"').replace(/&apos;/g, "'").replace(/[ \t]+/g, ' ');
  }
  if (e == 'pdf') {
    pdfjsLib.GlobalWorkerOptions.workerSrc =
      'https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.worker.min.js';
    const d = await pdfjsLib.getDocument({data: await f.arrayBuffer()}).promise;
    let out = '';
    for (let i = 1; i <= d.numPages; i++) {
      const c = await (await d.getPage(i)).getTextContent();
      out += c.items.map(x => x.str).join(' ') + '\n';
    }
    return out;
  }
  throw 0;   // unsupported format
}
```

### Module 2: Tokenizer and Word IDs (Java)

On the server the text is split on white space and tokens that contain no letter or digit are dropped. Each remaining word is converted to lower case, stripped of punctuation and mapped to an integer ID through a HashMap shared by the documents being compared, so that all later algorithms work on int arrays. The original words are kept so that the front end can show the text with highlights.

```
static Doc prep(String t, Map<String, Integer> dict) {
    Doc d = new Doc();
    for (String w : t.trim().split("\\s+")) if (w.matches(".*[A-Za-z0-9].*")) d.raw.add(w);
    d.ids = new int[d.raw.size()];
    for (int i = 0; i < d.ids.length; i++) {
        String key = d.raw.get(i).toLowerCase().replaceAll("[^a-z0-9]", "");
        Integer id = dict.get(key);
        if (id == null) { id = dict.size() + 3; dict.put(key, id); }
        d.ids[i] = id;
    }
    return d;
}

static int count(boolean[] m) { int c = 0; for (boolean b : m) if (b) c++; return c; }
```

### Module 3: Rabin-Karp, KMP and Z-function (Java)

Rabin-Karp computes a rolling polynomial hash for every window of three words: when the window moves, the oldest word is subtracted and the newest is added in O(1). Equal hashes are only candidates, so each one is verified. Web check verifies with KMP (a failure table followed by a single scan of the text), and Compare verifies with the Z-function applied to pattern + separator + text. A document that shares no verified three-word phrase with a source is skipped, so the costly suffix array is built only when needed.

```
/** Rabin-Karp: rolling hash of every k-word window -> hash -> start positions. */
static Map<Long, List<Integer>> rabinKarp(int[] a, int k) {
    Map<Long, List<Integer>> m = new HashMap<>();
```

```
    if (a.length < k) return m;
    final long M = 1_000_000_007L, B = 131;
    long h = 0, pw = 1;
    for (int i = 0; i < k - 1; i++) pw = pw * B % M;
    for (int i = 0; i < a.length; i++) {
        if (i >= k) h = ((h - a[i - k] * pw % M) % M + M) % M; // drop the oldest word
        h = (h * B + a[i]) % M;                                // add the newest word
        if (i >= k - 1) m.computeIfAbsent(h, x -> new ArrayList<>()).add(i - k + 1);
    }
    return m;
}

/** KMP: index of first occurrence of p in t, or -1. */
static int kmp(int[] t, int[] p) {
    int[] f = new int[p.length];
    for (int i = 1, k = 0; i < p.length; i++) {
        while (k > 0 && p[i] != p[k]) k = f[k - 1];
        if (p[i] == p[k]) k++;
        f[i] = k;
    }
    for (int i = 0, k = 0; i < t.length; i++) {
        while (k > 0 && t[i] != p[k]) k = f[k - 1];
        if (t[i] == p[k]) k++;
        if (k == p.length) return i - k + 1;
    }
    return -1;
}

/** Z-function: z[i] = length of the longest common prefix of s and s[i..]. */
static int[] zFunction(int[] s) {
    int n = s.length;
    int[] z = new int[n];
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = Math.min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}

/** Does p occur in t? (Z-function on p + separator + t) */
static boolean zContains(int[] t, int[] p) {
    int[] s = new int[p.length + 1 + t.length];
    System.arraycopy(p, 0, s, 0, p.length);
    s[p.length] = -1;
    System.arraycopy(t, 0, s, p.length + 1, t.length);
    int[] z = zFunction(s);
    for (int i = p.length + 1; i < s.length; i++) if (z[i] >= p.length) return true;
    return false;
}
```

**Module 4: Suffix Array, Kasai LCP Array and Match Marking (Java)**

The two word-ID arrays are joined as A + separator 1 + B + separator 2 (word IDs start at 3, so a separator never matches a word). The suffix array is built by prefix doubling and Kasai's algorithm computes the LCP array in linear time. Whenever two adjacent suffixes come from different documents and share at least K = 4 words, every word of that shared run is marked in both documents. The marks give the match percentage and the highlighted text.

```
/** Suffix array by prefix doubling, O(n log^2 n). */
static int[] suffixArray(int[] s) {
```

```
    int n = s.length;
    Integer[] sa = new Integer[n];
    int[] r = s.clone(), t = new int[n];
    for (int i = 0; i < n; i++) sa[i] = i;
    for (int k = 1; ; k <<= 1) {
        final int K = k;
        final int[] R = r;
        Comparator<Integer> c = (a, b) -> {
            if (R[a] != R[b]) return Integer.compare(R[a], R[b]);
            int x = a + K < n ? R[a + K] : -1, y = b + K < n ? R[b + K] : -1;
            return Integer.compare(x, y);
        };
        Arrays.sort(sa, c);
        t[sa[0]] = 0;
        for (int i = 1; i < n; i++) t[sa[i]] = t[sa[i - 1]] + (c.compare(sa[i - 1], sa[i]) <
0 ? 1 : 0);
        r = t.clone();
        if (r[sa[n - 1]] == n - 1) break;
    }
    int[] out = new int[n];
    for (int i = 0; i < n; i++) out[i] = sa[i];
    return out;
}

/** Kasai LCP: lcp[i] = common prefix length of suffixes sa[i-1] and sa[i]. */
static int[] lcp(int[] s, int[] sa) {
    int n = s.length;
    int[] rank = new int[n], l = new int[n];
    for (int i = 0; i < n; i++) rank[sa[i]] = i;
    for (int i = 0, h = 0; i < n; i++) {
        if (rank[i] > 0) {
            int j = sa[rank[i] - 1];
            while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
            l[rank[i]] = h;
            if (h > 0) h--;
        } else h = 0;
    }
    return l;
}

/** Finds shared passages (>= K words) between a and b. Returns {marksOfA, marksOfB}. */
static boolean[][] match(int[] a, int[] b, int K) {
    int n = a.length;
    int[] s = new int[n + 1 + b.length + 1];
    System.arraycopy(a, 0, s, 0, n);
    s[n] = 1;                                   // separators never equal a word id (ids >=
3)
    System.arraycopy(b, 0, s, n + 1, b.length);
    s[s.length - 1] = 2;
    int[] sa = suffixArray(s), l = lcp(s, sa);
    boolean[] A = new boolean[n], B = new boolean[b.length];
    for (int i = 1; i < s.length; i++) {
        if (l[i] < K) continue;
        int p = sa[i - 1], q = sa[i];
        if (p > q) { int tmp = p; p = q; q = tmp; }
        if (p < n && q > n)
            for (int j = 0; j < l[i]; j++) { A[p + j] = true; B[q - n - 1 + j] = true; }
    }
    return new boolean[][]{A, B};
}
```

**Module 5: Server, Web Check and Compare Engine (Java)**

The Java server (com.sun.net.httpserver) serves index.html and two endpoints. /api/web compares the submitted document with every source in the built-in index: Rabin-Karp finds candidates, KMP verifies them and the suffix array with LCP marks the shared passages. Sources are ranked by percentage and the union of the marked words gives the overall percentage. /api/compare works in the same way for two uploaded documents, using the Z-function for verification. Both return JSON containing the percentages, the words and a 0/1 mark for every word.

```
public static void main(String[] args) throws Exception {
    HttpServer s = HttpServer.create(new InetSocketAddress(8080), 0);
    s.createContext("/", e -> {
        String p = e.getRequestURI().getPath();
        if (p.equals("/") || p.equals("/index.html")) send(e, 200, "text/html;
charset=utf-8", new String(Files.readAllBytes(Paths.get("index.html")),
StandardCharsets.UTF_8));
        else send(e, 404, "text/plain", "Not found");
    });
    s.createContext("/api/web", e -> send(e, 200, "application/json; charset=utf-8",
web(form(e))));
    s.createContext("/api/compare", e -> send(e, 200, "application/json; charset=utf-8",
compare(form(e))));
    s.start();
    System.out.println("Plagiarism Checker running at http://localhost:8080");
}

static String web(Map<String, String> f) {
    Map<String, Integer> dict = new HashMap<>();
    Doc d = prep(f.getOrDefault("text", ""), dict);
    int n = d.ids.length;
    if (n < 8) return "{\"error\":\"Please add at least 8 words.\"}";
    Map<Long, List<Integer>> dh = Algorithms.rabinKarp(d.ids, 3);
    boolean[] union = new boolean[n];
    List<Res> res = new ArrayList<>();
    for (String[] s : SRC) {
        Doc x = prep(s[2], dict);
        Map<Long, List<Integer>> sh = Algorithms.rabinKarp(x.ids, 3);
        int ok = 0;
        for (Map.Entry<Long, List<Integer>> en : dh.entrySet()) {          // Rabin-Karp
finds candidates
            if (!sh.containsKey(en.getKey())) continue;
            int p = en.getValue().get(0);
            if (Algorithms.kmp(x.ids, Arrays.copyOfRange(d.ids, p, p + 3)) >= 0) ok++; // KMP
verifies
        }
        Res r = new Res(); r.name = s[0]; r.url = s[1];
        if (ok > 0) {
            boolean[] a = Algorithms.match(d.ids, x.ids, 4)[0];             // suffix array +
LCP
            for (int i = 0; i < n; i++) if (a[i]) union[i] = true;
            r.pct = Math.round(100f * count(a) / n);
        }
        res.add(r);
    }
    res.sort((p, q) -> q.pct - p.pct);
    StringBuilder b = new StringBuilder("{\"total\":" + Math.round(100f * count(union) / n) +
",\"sources\":[");
    for (int i = 0; i < res.size(); i++) {
        if (i > 0) b.append(',');
```

```
        b.append("{\"name\":" + q(res.get(i).name) + ",\"url\":" + q(res.get(i).url) +
",\"pct\":" + res.get(i).pct + "}");
    }
    return
b.append("],\"words\":").append(words(d.raw)).append(",\"marks\":").append(marks(union)).appe
nd('}').toString();
}

static String compare(Map<String, String> f) {
    Map<String, Integer> dict = new HashMap<>();
    Doc a = prep(f.getOrDefault("a", ""), dict), b = prep(f.getOrDefault("b", ""), dict);
    if (a.ids.length < 8 || b.ids.length < 8) return "{\"error\":\"Add at least 8 words to
both documents.\"}";
    Map<Long, List<Integer>> ha = Algorithms.rabinKarp(a.ids, 3), hb =
Algorithms.rabinKarp(b.ids, 3);
    boolean any = false;
    for (Map.Entry<Long, List<Integer>> en : ha.entrySet()) {
        if (!hb.containsKey(en.getKey())) continue;
        int p = en.getValue().get(0);
        if (Algorithms.zContains(b.ids, Arrays.copyOfRange(a.ids, p, p + 3))) { any = true;
break; } // Z-function verifies
    }
    boolean[][] m = any ? Algorithms.match(a.ids, b.ids, 4) : new boolean[][]{new
boolean[a.ids.length], new boolean[b.ids.length]};
    int ca = count(m[0]), cb = count(m[1]);
    return "{\"overall\":" + Math.round(100f * (ca + cb) / (a.ids.length + b.ids.length))
        + ",\"pa\":" + Math.round(100f * ca / a.ids.length) + ",\"pb\":" + Math.round(100f *
cb / b.ids.length)
        + ",\"aw\":" + words(a.raw) + ",\"am\":" + marks(m[0]) + ",\"bw\":" + words(b.raw) +
",\"bm\":" + marks(m[1]) + "}";
}
```

### Module 6: Login, Sign up and History (JavaScript)

Sign up stores the name against the roll number, and login checks that the name matches the roll number that was registered. Each check is saved to the user's history (newest first, last 50 entries) in the browser's localStorage under the roll number. The front-end code that sends a Web check request and saves the result is also shown (rendering code is abridged).

```
$('#go').onclick = () => {
  const n = $('#un').value.trim(), r = $('#ur').value.trim().toUpperCase();
  if (!n || !r) { $('#er').textContent = 'Please enter your name and roll number.'; return; }
  if (mode == 'S') {                                  // sign up
    if (users[r]) { $('#er').textContent = 'This roll number already has an account. Please
login.'; return; }
    users[r] = n; ST.s('pc_users', users);
  } else if (!users[r] || users[r].toLowerCase() != n.toLowerCase()) {   // login
    $('#er').textContent = 'No matching account found. Check your details or sign up.';
return;
  }
  me = {n: users[r], r};
  hist = ST.g('pc_h_' + r, []);                       // this user's saved history
  // ... hide the login page, show the application ...
};

function save(type, title, score, top) {
  hist.unshift({type, title, score, top, d: new Date().toLocaleString()});
  hist = hist.slice(0, 50);                           // newest first, keep 50
  ST.s('pc_h_' + me.r, hist);                         // localStorage
  renderH();
}
```

```
async function post(u, o) {
  const r = await fetch(u, {method: 'POST',
    headers: {'Content-Type': 'application/x-www-form-urlencoded'},
    body: new URLSearchParams(o)});
  if (!r.ok) throw 0;
  return r.json();
}

$('#bw').onclick = async () => {                        // Web check button
  const r = await post('/api/web', {text: $('#tw').value});
  if (r.error) { toast(r.error); return; }
  // ... draw the match ring, the ranked r.sources and hl(r.words, r.marks) ...
  save('web', title, r.total, r.sources[0].pct ? r.sources[0].name : 'No matches');
};
```

**User interface flow**

After signing in, the user chooses Web check, Compare or History from the top bar. For Web check the text (typed, pasted or extracted from a file) is posted to /api/web and the page shows a percentage ring, a ranked bar list of the sources and the text with the matched words highlighted. For Compare, the two texts are posted to /api/compare and both documents are shown side by side with highlights. Each check is added to the user's history. A switch in the top bar changes between light and dark mode, and the 3D cube and cards react to the mouse.

# 7. RESULTS & DISCUSSION

The Java back end was tested by sending requests to /api/compare and /api/web. Compare mode used three short documents: Documents 1 and 2 are two similar paragraphs and Document 3 is an unrelated text about baking. Web check used three texts that were checked against the built-in source index.

**Test Input**

**Document 1:** “Artificial neural networks are computational models inspired by biological neural networks. They consist of input, hidden, and output layers where weights are iteratively updated using gradient descent algorithms to minimize loss functions.”

**Document 2:** “Artificial neural networks are computational models inspired by biological neural networks. They consist of connected input, hidden, and output layers where weights are calculated using gradient descent to reduce loss values.”

**Document 3:** “The recipe calls for flour, butter and sugar, mixed gently before baking in a hot oven for twenty minutes.”

**Text W1:** “Photosynthesis is the process by which green plants and some bacteria convert light energy into chemical energy. During this process carbon dioxide and water are transformed into glucose and oxygen inside the chloroplasts of the cell. This is why forests matter for our planet.”

**Text W2:** “My grandmother bakes bread every Sunday morning and the whole street smells wonderful until noon.”

**Text W3:** “Climate change refers to long term shifts in temperatures and weather patterns. Since the nineteenth century human activities have been the main driver of climate change, mostly through burning fossil fuels. Photosynthesis is the process by which green plants convert light energy into chemical energy, and it is essential for life on our planet.”

**Program Output**

```
Compare mode (POST /api/compare)
Words: Document 1=32, Document 2=31, Document 3=19
Document 1 x Document 2: match=70%  (in Document 1=69%, in Document 2=71%)
Document 1 x Document 3: match=0%  (in Document 1=0%, in Document 3=0%)
Document 2 x Document 3: match=0%  (in Document 2=0%, in Document 3=0%)

Document 1 highlighted ([ ]=shared passage):
[Artificial] [neural] [networks] [are] [computational] [models] [inspired] [by] [biological]
[neural] [networks.] [They] [consist] [of] [input,] [hidden,] [and] [output] [layers] [where]
[weights] [are] iteratively updated using gradient descent algorithms to minimize loss
functions.

Document 2 highlighted:
```

```
[Artificial] [neural] [networks] [are] [computational] [models] [inspired] [by] [biological]
[neural] [networks.] [They] [consist] [of] connected [input,] [hidden,] [and] [output]
[layers] [where] [weights] [are] calculated using gradient descent to reduce loss values.

Web check mode (POST /api/web)
Text W1 (44 words): overall match=82%  | Wikipedia 82%, Britannica 0%, Khan Academy 0%,
others 0%
Text W2 (15 words): overall match=0%   | no source above 0%
Text W3 (54 words): overall match=74%  | Britannica 48%, Wikipedia 26%, others 0%

Text W3 highlighted:
[Climate] [change] [refers] [to] [long] [term] [shifts] [in] [temperatures] [and] [weather]
[patterns.] [Since] [the] [nineteenth] [century] [human] [activities] [have] [been] [the]
[main] [driver] [of] [climate] [change,] mostly through burning fossil fuels.
[Photosynthesis] [is] [the] [process] [by] [which] [green] [plants] [convert] [light]
[energy] [into] [chemical] [energy,] and it is essential for life on our planet.
```

*Figure 7.1 Output of the Java back end for the sample documents*

**Explanation:** Documents 1 and 2 share the first sentence and the phrase “input, hidden, and output layers where weights are”, which the suffix array and LCP array mark as shared passages (69% of Document 1 and 71% of Document 2, 70% overall). The rest of the second sentence is reworded (“iteratively updated using gradient descent algorithms to minimize loss” versus “calculated using gradient descent to reduce loss”); no four consecutive words are common, so it is not counted. The unrelated baking document scores 0% against both. In Web check, Text W1 is a lightly extended copy of the Wikipedia passage in the index, so 82% of its words are marked and Wikipedia is ranked first. Text W2 is original and scores 0%. Text W3 combines a Britannica-style sentence and a Wikipedia-style sentence and is reported as 74% overall, with Britannica as the most similar source (48%).

## Application Screenshots

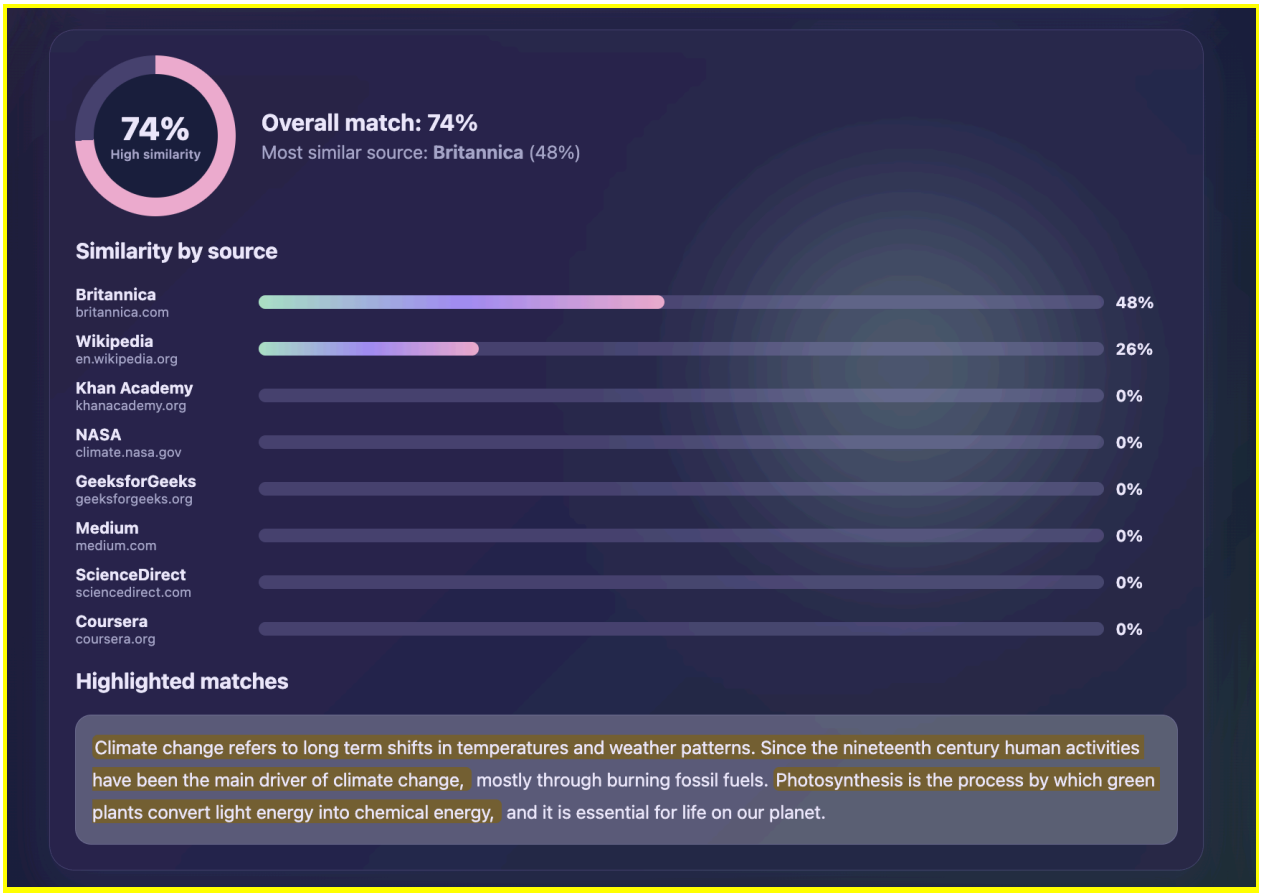



*Figure 7.2 Web check result with ranked sources and highlighted matches*

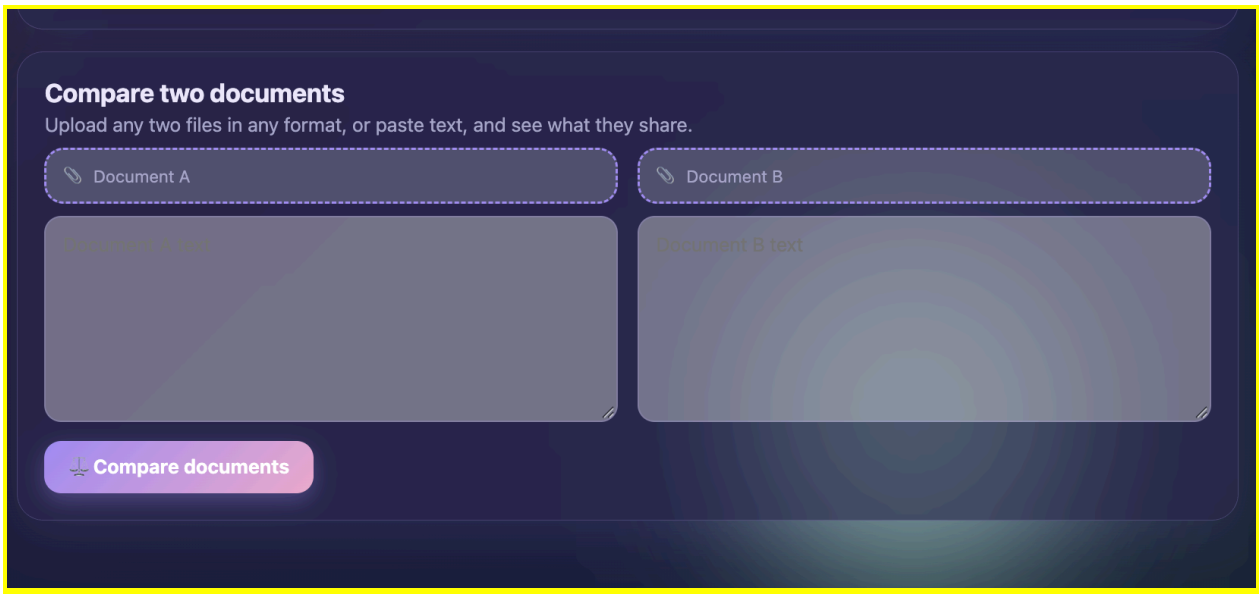



*Figure 7.3 Compare result for two uploaded documents*

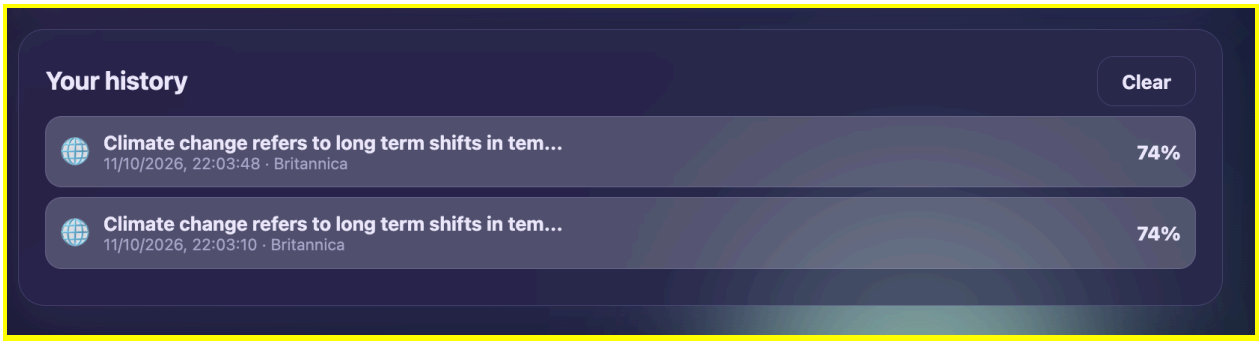



*Figure 7.4 history pages*

**Discussion**

- Shared passages are located exactly, with their positions in both documents, which allows precise highlighting of the copied words.
- Rabin-Karp hashing with KMP or Z-function verification works as a cheap filter: a source that shares no verified three-word phrase with the document is rejected without building a suffix array, so unrelated texts score 0% quickly.
- Only runs of four or more consecutive words are counted. This keeps false matches low, but a reworded passage with no four-word run in common, as in the second sentence of the test, is not detected.
- In Web check the percentage is measured against the submitted document, and in Compare each document gets its own percentage, so the two values can differ.

documents.

# 8. CONCLUSION & FUTURE SCOPE

The project met its objective of building a plagiarism checker for short documents with a Java back end and a browser front end. It extracts text from PDF, Word, PowerPoint and text files, filters candidate phrases with a Rabin-Karp rolling hash verified by KMP and the Z-function, and finds all shared passages of four or more words using a suffix array and Kasai's LCP array. The results are presented as a match percentage with highlighted text, a ranking of the most similar sources in Web check mode, and a side-by-side view in Compare mode. Login with name and roll number and a saved history make it practical for repeated use. Tests on sample documents showed correct detection of copied text and a 0% score for unrelated text.

**Future Scope.** The system can be extended in the following ways:

**Near term**

- Replace the built-in source index with real web search through a search API and fetch the result pages, so that documents are checked against the live internet.
- Ignore stop words when computing the percentage and build the suffix array with radix sort to reach O(n log n) for longer documents.

**Medium term**

- Add paraphrase detection with an edit-distance based dynamic-programming method that models reordering and word substitution.
- Use winnowing fingerprints to filter unrelated sources quickly when many sources or documents are compared.

**Long term**

- Identify the minimal set of source documents that explains a submission using a set-cover or network-flow formulation.
- Store accounts and history in a database (for example MongoDB) with secure authentication, and add exportable PDF reports.

## 9. REFERENCES


1. S. Schleimer, D. S. Wilkerson and A. Aiken, “Winnowing: Local Algorithms for Document Fingerprinting,” Proc. ACM SIGMOD, 2003. https://theory.stanford.edu/~aiken/publications/papers/sigmod03.pdf

2. A. Aiken, “MOSS: A System for Detecting Software Similarity,” Stanford University. https://theory.stanford.edu/~aiken/moss/

3. S. Brin, J. Davis and H. Garcia-Molina, “Copy Detection Mechanisms for Digital Documents,” Proc. ACM SIGMOD, 1995. https://doi.org/10.1145/223784.223855

4. U. Manber and G. Myers, “Suffix Arrays: A New Method for On-Line String Searches,” SIAM J. Computing, 22(5), 1993. https://doi.org/10.1137/0222058

5. T. Kasai, G. Lee, H. Arimura, S. Arikawa and K. Park, “Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications,” CPM 2001. https://doi.org/10.1007/3-540-48194-X_17

6. R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” IBM Journal of Research and Development, 31(2), 1987. https://doi.org/10.1147/rd.312.0249

7. D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM J. Computing, 6(2), 1977. https://doi.org/10.1137/0206024

8. D. Gusfield, Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.

9. A. Z. Broder, “On the Resemblance and Containment of Documents,” Proc. Compression and Complexity of Sequences, 1997. https://doi.org/10.1109/SEQUEN.1997.666900

10. Turnitin, “Similarity Report.” https://www.turnitin.com

11. Mozilla, “PDF.js.” https://mozilla.github.io/pdf.js/

12. S. Knightley and contributors, “JSZip.” https://stuk.github.io/jszip/